MICROSERVICES

Build · Dockerize · Deploy · Test

Item Service	Order Service	Payment Service	API Gateway

Port 8081 • Port 8082 • Port 8083 • Gateway: 8080

1. Lab Overview & Objectives

In this 2-hour practical lab you will build a complete Microservices system from scratch. You are free to choose any supported technology stack for each service the only requirement is that each service runs in Docker and can be reached through the API Gateway.

Learning Outcomes

- Understand how microservices communicate through an API Gateway
- Apply containerisation with Docker and Docker Compose
- Design and expose REST API endpoints across independent services
- Test multi-service systems using Postman
- Practise polyglot development — mixing frameworks and languages

> *You are NOT required to use Spring Boot for every service. See Section 2 for supported technology options per service.*

2. Technology Stack Options

You may mix and match technologies across your services. Each service runs independently in its own container, so they do not need to share the same language or framework. Choose what you are most comfortable with.

Service	Option A	Option B	Option C	Language
Item Service	Spring Boot	Node.js + Express	FastAPI (Python)	Any
Order Service	Spring Boot	Gin (Go)	Django REST	Any
Payment Service	Spring Boot	Flask (Python)	.NET Core	Any
API Gateway	Spring Cloud Gateway	Kong / NGINX	Express Gateway	Any

> **Tip:** If you are unsure, use Spring Boot for all services — it is the safest choice and well-documented examples are provided throughout this lab sheet.

Quick Start Templates

Option A — Spring Boot (Java)

Generate your project at: https://start.spring.io

- Dependencies to add: Spring Web
- Java version: 17 or 21
- Build tool: Maven

Option B — Node.js + Express

```
mkdir item-service && cd item-service
npm init -y
npm install express
# Create index.js — see code example below
```

Option C — FastAPI (Python)

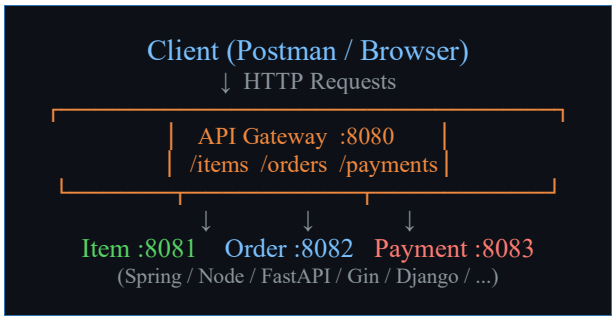
```
mkdir payment-service && cd payment-service
pip install fastapi uvicorn
# Create main.py — see code example below
```

Option D — Gin (Go)

```
mkdir order-service && cd order-service
go mod init order-service
go get github.com/gin-gonic/gin
```

3. System Architecture

The diagram below shows how all four components connect. The API Gateway is the single entry point — it routes every request to the correct microservice. Services do NOT call each other directly.



Routing Rules

The API Gateway must route requests based on the URL path prefix:

Incoming URL	Routes To	Target
http://localhost:8080/items/**	Item Service	http://item-service:8081
http://localhost:8080/orders/**	Order Service	http://order-service:8082
http://localhost:8080/payments/**	Payment Service	http://payment-service:8083

4. Implementation Guide & Code Examples

The following examples show a complete controller for each major framework option. You only need to implement the tech stack you chose in Section 2.

4.1 Item Service

Spring Boot (Java)

```
@RestController
@RequestMapping("/items")
public class ItemController {

    // Simple in-memory list (no database needed)
    private List<String> items = new ArrayList<>(List.of("Book", "Laptop", "Phone"));

    @GetMapping
    public List<String> getItems() {
        return items;
    }

    @PostMapping
    public ResponseEntity<String> addItem(@RequestBody String item) {
        items.add(item);
        return ResponseEntity.status(HttpStatus.CREATED).body("Item added: " + item);
    }
```

```
    @GetMapping("/{id}")
    public ResponseEntity<String> getItem(@PathVariable int id) {
        if (id < 0 || id >= items.size())
            return ResponseEntity.notFound().build();
        return ResponseEntity.ok(items.get(id));
    }
}
```

4.2 Order Service

Spring Boot (Java)

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    private List<Map<String,Object>> orders = new ArrayList<>();
    private int idCounter = 1;

    @GetMapping
    public List<Map<String,Object>> getOrders() {
        return orders;
    }

    @PostMapping
    public ResponseEntity<Map<String,Object>> placeOrder(@RequestBody Map<String,Object> order) {
        order.put("id", idCounter++);
        order.put("status", "PENDING");
        orders.add(order);
        return ResponseEntity.status(201).body(order);
    }

    @GetMapping("/{id}")
    public ResponseEntity<?> getOrder(@PathVariable int id) {
        return orders.stream()
            .filter(o -> o.get("id").equals(id))
            .findFirst()
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}
```

4.3 Payment Service

Spring Boot (Java)

```
@RestController
@RequestMapping("/payments")
public class PaymentController {

    private List<Map<String,Object>> payments = new ArrayList<>();
    private int idCounter = 1;

    @GetMapping
    public List<Map<String,Object>> getPayments() {
        return payments;
    }
```

```
    @PostMapping("/process")
    public ResponseEntity<Map<String,Object>> processPayment(@RequestBody Map<String,Object> payment) {
        payment.put("id", idCounter++);
        payment.put("status", "SUCCESS");
        payments.add(payment);
        return ResponseEntity.status(201).body(payment);
    }

    @GetMapping("/{id}")
    public ResponseEntity<?> getPayment(@PathVariable int id) {
        return payments.stream()
            .filter(p -> p.get("id").equals(id))
            .findFirst()
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}
```

4.4 API Gateway

Spring Cloud Gateway (application.yml)

Add the Spring Cloud Gateway dependency in pom.xml, then configure routing in application.yml:

```
server:
  port: 8080

spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      routes:
        - id: item-service
          uri: http://item-service:8081
          predicates:
            - Path=/items/**

        - id: order-service
          uri: http://order-service:8082
          predicates:
            - Path=/orders/**

        - id: payment-service
          uri: http://payment-service:8083
          predicates:
            - Path=/payments/**
```

5. Docker & Docker Compose

5.1 Dockerfiles

Spring Boot — Dockerfile

```
FROM eclipse-temurin:17-jdk-alpine
WORKDIR /app
COPY target/*.jar app.jar
ENTRYPOINT ["java","-jar","app.jar"]
```

5.2 Docker Compose

Create docker-compose.yml in the root of your project. Adjust the build paths to match your folder names:

```
version: '3.8'

services:
  item-service:
    build: ./item-service
    container_name: item-service
    ports:
      - "8081:8081"
    networks:
      - microservices-net

  order-service:
    build: ./order-service
    container_name: order-service
    ports:
      - "8082:8082"
    networks:
      - microservices-net

  payment-service:
    build: ./payment-service
    container_name: payment-service
    ports:
      - "8083:8083"
    networks:
      - microservices-net

  api-gateway:
    build: ./api-gateway
    container_name: api-gateway
    ports:
      - "8080:8080"
    depends_on:
      - item-service
      - order-service
      - payment-service
    networks:
      - microservices-net

networks:
  microservices-net:
    driver: bridge
```

Important: All services are on the same Docker network (microservices-net). Use the service name (e.g. item-service) as the hostname inside your gateway config — NOT localhost.

5.3 Build & Run Commands

```
# Build all services (run from project root)
docker-compose build

# Start all containers
docker-compose up

# Start in background (detached mode)
docker-compose up -d

# View running containers
docker ps

# View logs for a specific service
docker-compose logs item-service

# Stop and remove all containers
docker-compose down
```

6. Required API Endpoints

All endpoints below must work when called through the API Gateway on port 8080. Minimum requirement is the GET endpoints. POST endpoints are required for the bonus marks.

Method	Endpoint	Service	Description
GET	/items	Item	Returns list of all items
POST	/items	Item	Create a new item
GET	/items/{id}	Item	Get item by ID
GET	/orders	Order	Returns all orders
POST	/orders	Order	Place a new order
GET	/orders/{id}	Order	Get order by ID
GET	/payments	Payment	Returns all payments
POST	/payments/process	Payment	Process a payment
GET	/payments/{id}	Payment	Get payment status

Sample Request Bodies (for POST endpoints)

POST /items

```
{
  "name": "Headphones"
}
```

POST /orders

```
{
  "item": "Laptop",
  "quantity": 2,
  "customerId": "C001"
}
```

POST /payments/process

```
{
  "orderId": 1,
  "amount": 1299.99,
  "method": "CARD"
}
```

7. Testing with Postman

1. Open Postman (or install from https://postman.com)
2. Create a new Collection called "Microservices Lab"
3. Add a new request for each endpoint listed below
4. Make sure docker-compose is running before testing

8. Submission

Students must submit one public GitHub repository that contains the complete working microservices system, including Docker setup and API testing evidence. Include this repo link in word document and submit it.